PAK-AUSTRIA FACHHOCHSCHULE:
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

Programming Fundamentals Lab 03	**Instructor: Mr. Muhammad Hamza**
Program: AI 2025	**Date: 23rd October, 2025**
Marks: 10	**Lab Engineer: Qandil Fatima**
Student Name: Eshal Ayub	**Reg. No: B25F1061AI100**

Instructions:

- Submit the file with your names following your registration numbers like **AI001_Name.**
- **Copy Paste work will be given no marks.**

Lab Task

1. Assume that the following variables are defined: a-> A
   int age;
   double pay;
   char section;

Write a single cin statement that will read input into each of these variables.

Untitled1.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    int age;
    double pay;
    char section;

    cout << "Enter age, pay, and section: ";
    cin >> age >> pay >> section;

    cout << "\nAge: " << age << endl;
    cout << "Pay: " << pay << endl;
    cout << "Section: " << section << endl;

    return 0;
}
```

Explanation:

- cin >> age >> pay >> section; reads all three inputs **in one line**.

Output:

C:\Users\DELL PRECision 7550

```
Enter age, pay, and section: 456
666
777
Age: 456
Pay: 666
Section: 7

--------------------------------
Process exited after 12.73 seconds with return value 0
Press any key to continue . . .
```

2. Complete the following table by writing the value of each expression in the Value column according C++ language rules.

Expressions	Value
28 / 4 – 2	5 (28 ÷ 4 = 7 → 7 − 2 = **5)**
6 + 12 * 2 – 8	22(12 × 2 = 24 → 6 + 24 − 8 = **22)**
4 + 8 * 2	20 (8 × 2 = 16 → 4 + 16 = **20)**
6 + 17 % 3 – 2	5(17 % 3 = 2 → 6 + 2 − 2 = **6**, → 6 + 2 − 2 = **6**)
2 + 22 * (9 – 7)	46 ((9 – 7) = 2 → 22 × 2 = 44 → 2 + 44 = **46**)
(8 + 7) * 2	30 ((8 + 7) = 15 → 15 × 2 = **30**)
(16 + 7) % 2 – 1	0((16 + 7) = 23 → 23 % 2 = 1 → 1 − 1 = **0**)
12 / (10 – 6)	3 ((10 − 6) = 4 → 12 ÷ 4 = **3**)
(19 – 3) * (2 + 2) / 4	16 ((19 − 3) = 16, (2 + 2) = 4 → 16 × 4 = 64 → 64 ÷ 4 = **16**)

3. Assume a program has the following variable definitions:

```
int units;
float mass;
double weight;
weight = mass * units;
```

Which automatic data type conversion will take place?

A. mass is demoted to an int, units remains an int, and the result of mass * units is an int.

B. units is promoted to a float, mass remains a float, and the result of mass * units is a float.

C. units is promoted to a float, mass remains a float, and the result of mass * units is a double.

Answer:

C. units are promoted to a float, mass remains a float, and the result of mass × units is double.

Explanation:
When mass (a float) is multiplied by units (an int), C++ automatically converts the integer to a float before performing the operation. This makes the result of mass * units float. Then, since that result is being assigned to weight (a double), it is automatically promoted from float to double.

So the conversion happens in two steps:

1. units → promoted to **float.**
2. Result (**float**) → promoted to **double** when stored in weight.

---

4. Assume a program has the following variable definitions:

int a, b = 2;

float c = 4.2;

and the following statement: a = b * c;

What value will be stored in a?

A. 8.4 B. 8 C. 0 D. None of the above

Answer:

Correct Answer: B. 8

Explanation:
In C++, when an int and a float are multiplied, the int value is automatically converted to a float before the operation.
So the expression b * c becomes 2 * 4.2, which equals **8.4** (a float value).

However, since the result is being assigned to a, which is an **int**, the decimal part (.4) is **truncated**, not rounded.
Therefore, only the whole number part (8) is stored in a.

---

5. Assume that qty and salesReps are both integers. Use a type cast expression to rewrite the following statement so it will no longer perform integer division.

unitsEach = qty / salesReps;

Answer:
unitsEach = (float)qty / salesReps;

Explanation:
Since both qty and salesReps are integers, dividing them directly will perform integer division, which removes any fractional part of the result.
To fix this, we use type casting to temporarily treat one of the integers (here, qty) as a float. This ensures the division is done in floating-point mode, keeping the decimal portion in the final answer.

---

6. Write a program that can be used as a math tutor for a young student. The program should display two random numbers to be added, such as

```
  247
+ 129
---------
```

The program should then pause while the student works on the problem. When the student is ready to check the answer, he or she can press a key and the program will display the correct solution:

```
  247
+ 129
--------
376
```

[*] Untitled1

```cpp
#include <iostream>
#include <cstdlib>   // For rand() and srand()
#include <ctime>     // For time()
using namespace std;

int main() {
    int number1, number2, answer;

    // Seed random number generator
    srand(time(0));

    // Generate two random numbers between 100 and 999
    number1 = rand() % 900 + 100;
    number2 = rand() % 900 + 100;

    // Display the problem
    cout << "  " << number1 << endl;
    cout << "+ " << number2 << endl;
    cout << "---------" << endl;
    cout << "Press Enter to see the answer...";
    cin.get();  // Wait for user to press Enter

    // Calculate and display the correct answer
    answer = number1 + number2;
    cout << "  " << number1 << endl;
    cout << "+ " << number2 << endl;
    cout << "---------" << endl;
    cout << "  " << answer << endl;

    return 0;
}
```

Output:

C:\Users\DELL PRECision 7550

```
  667
+ 190
---------
Press Enter to see the answer...
  667
+ 190
---------
  857

--------------------------------
Process exited after 4.449 seconds with return value 0
Press any key to continue . . .
```

Explanation:

rand() generates random numbers, and srand(time(0)) ensures different numbers each time the program runs.

- The range rand() % 900 + 100 creates **three-digit numbers** (100–999).
- cin.get() pauses the program until the user presses Enter.
- Finally, it calculates and displays the sum neatly formatted.

7. Each of the following programs has some errors. Locate as many as you can.

Program-1

```
using namespace std;
void  main ()
{
}
double number1, number2, sum;
cout << "Enter a number: ";
Cin << number1;
cout << "Enter another number: ";
cin << number2;
number1 + number2 = sum;
cout "The sum of the two numbers is " << sum
```

Errors & Fixes Explained

1. **Missing `#include <iostream>`**
   → Needed for `cout` and `cin` to work.
2. **`void main()` is invalid in standard C++**
   → Should be `int main()` and end with `return 0;`
3. **Variable declarations are outside the function**
   → Move `double number1, number2, sum;` **inside `main()`**.
4. **Wrong operator used for input**
   → `cin` uses `>>`, not `<<`.
5. **`Cin` is incorrectly capitalized**
   → C++ is case-sensitive; it should be `cin`.
6. **Assignment statement is backwards**
   → Should be `sum = number1 + number2;`.
7. **Missing `<<` after `cout`**
   → Needs `cout << "The sum of the two numbers is " << sum;`.
8. **Missing semicolon (;)**
   → At the end of the last `cout` line.

Corrected Program:

Untitled1.cpp Untitled2.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    double number1, number2, sum;

    cout << "Enter a number: ";
    cin >> number1;

    cout << "Enter another number: ";
    cin >> number2;

    sum = number1 + number2;

    cout << "The sum of the two numbers is " << sum << endl;

    return 0;
}
```

Output:

```
C:\Users\DELL PRECision 7550

Enter a number: 7
Enter another number: 8
The sum of the two numbers is 15

--------------------------------
Process exited after 5.386 seconds with return value 0
Press any key to continue . . .
```

Program-2

```cpp
#include <iostream>

using namespace std;

void main()

{

int number1, number2;

float quotient;

cout << "Enter two numbers and I will divide\n";

cout << "the first by the second for you.\n";

cin >> number1, number2;
```

quotient = float<static_cast>(number1) / number2;

cout << quotient

}

Errors and Fixes:

1) **void main() is not standard**
   → Should be int main() and must end with return 0;.
2) **Incorrect input statement:**

   cin >> number1, number2;

   This doesn't take both inputs.
   → Should be: cin >> number1 >> number2;

3) **Wrong type casting syntax:**

   float<static_cast>(number1)
   This is invalid in C++.
   → Correct form: static_cast<float>(number1)

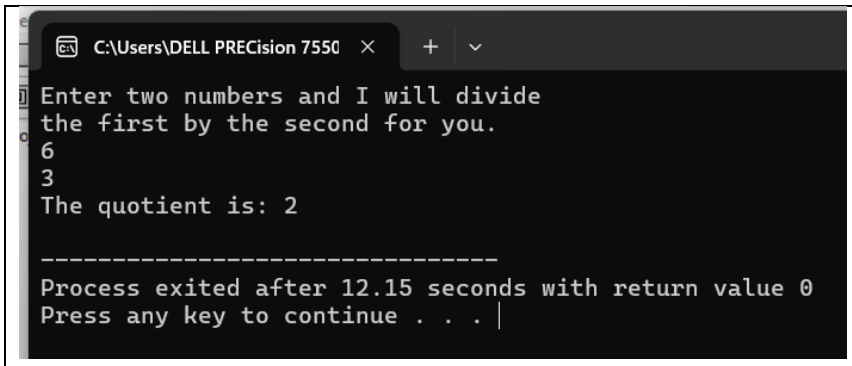
4) **Missing semicolon after cout << quotient**
   → Add (;) at the end.
5) **Optional but recommended:**
   Add endl for cleaner output.

Corrected Program

Untitled1.cpp | Untitled2.cpp | Untitled3.cpp | [*] Untitled4

```
#include <iostream>
using namespace std;

int main()
{
    int number1, number2;
    float quotient;

    cout << "Enter two numbers and I will divide\n";
    cout << "the first by the second for you.\n";

    cin >> number1 >> number2;

    quotient = static_cast<float>(number1) / number2;

    cout << "The quotient is: " << quotient << endl;

    return 0;
}
```

Output:



Bonus Task:

Write a C++ program that starts with two integers: a = 5 and b = 10. Show what happens when you use pre and post increment/decrement in simple expressions, then decide which value is greater using if–else. Specifically:

print the result of ++a + b,

print the result of a++ + --b,

print the result of --a + b--,

after these lines, print the final values of a and b,

use if–else to print “a is greater than b”, “b is greater than a”, or “a equals b”.

```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 10;

    cout << "Initial values: a = " << a << ", b = " << b << endl;

    cout << "\nResult of ++a + b = " << ++a + b << endl;   // Pre-increment a, then add b
    cout << "Result of a++ + --b = " << a++ + --b << endl; // Use current a, then increment; pre-decrement b
    cout << "Result of --a + b-- = " << --a + b-- << endl; // Pre-decrement a; use b, then post-decrement

    cout << "\nFinal values after all operations:" << endl;
    cout << "a = " << a << ", b = " << b << endl;

    // Compare values
    if (a > b)
        cout << "a is greater than b" << endl;
    else if (b > a)
        cout << "b is greater than a" << endl;
    else
        cout << "a equals b" << endl;

    return 0;
}
```

Output:

```
C:\Users\DELL PRECision 7550

Initial values: a = 5, b = 10

Result of ++a + b = 16
Result of a++ + --b = 15
Result of --a + b-- = 15

Final values after all operations:
a = 6, b = 8
b is greater than a

--------------------------------
Process exited after 0.1058 seconds with return value 0
Press any key to continue . . .
```

Expression	Description	Calculation	New a	New b
++a + b	Pre-increment a (a = 6), then 6 + 10	= **16**	6	10
a++ + --b	Use a (6) then a = 7; pre-decrement b = 9	6 + 9 = **15**	7	9
--a + b--	Pre-decrement a = 6; use b (9), then b = 8	6 + 9 = **15**	6	8

So, **final values**:

a = 6
b = 8

Then the program prints:

b is greater than a